AI Forward Deployed Engineer

The AI Architect & FDE Course



Week 2 Day 2

3 weeks to Forward Deployed Engineer

Theory Practical Projects

Week 3: Become an AI FDE

- AI Solutions Architect
- Security, Guardrails, API Gateways
- Enterprise Context Layer & Integrations
- Production Deployment
- Capstone

Week 2: Become an AI Architect

- Architecture, Evals, Observability
- Agentic RAG, MCP, Memory
- Durable State & Human in the Loop
- Agent Teams, Swarms, Fleet
- Self-Improvement, RSI

Week 1: Become an AI Engineer — AI ENGINEER

- Path to FDE
- Frontier LLMs & Agents
- Coding Agents & ADLC
- Open-Source, Local, Token Cost
- Harness Engineering

Taking stock

You can already:

- [x] Describe the role of the AI Forward Deployed Engineer
- [x] Build agents with LangChain and choose the right model for a task
- [x] Explain Harness Engineering and build a harness for a task
- [x] Use evaluations and observability in Langfuse to drive agent design decisions

By the end of today, you'll be able to:

- [ ] Define RAG and Agentic RAG
- [ ] Lay out MCP architecture and compare MCP transport mechanisms
- [ ] Decide whether to use MCP with your agent
- [ ] Create an agent with long-term memory with Agentic RAG over MCP

Recap

Drive architecture decisions based on evals to solve a business problem



The confusion matrix and the cost matrix

Business appetite for false positives and false negatives may differ



Have a suite of metrics for a run

Accuracy, kappa, precision, recall and the straight-through rate



Langfuse is great for experiments

Track datasets and compare scores

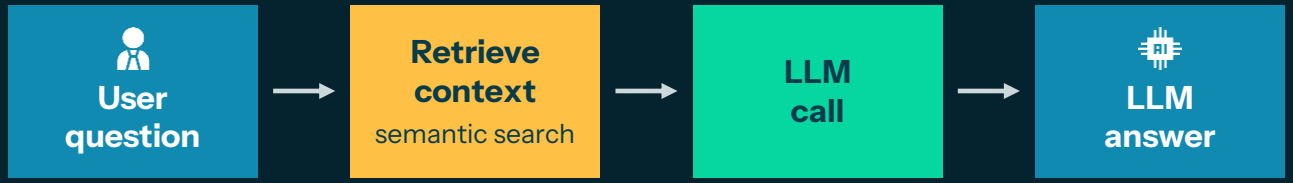


Business logic belongs in code

Prove it with evals

Retrieval Augmented Generation

Search for relevant context and add it to the prompt



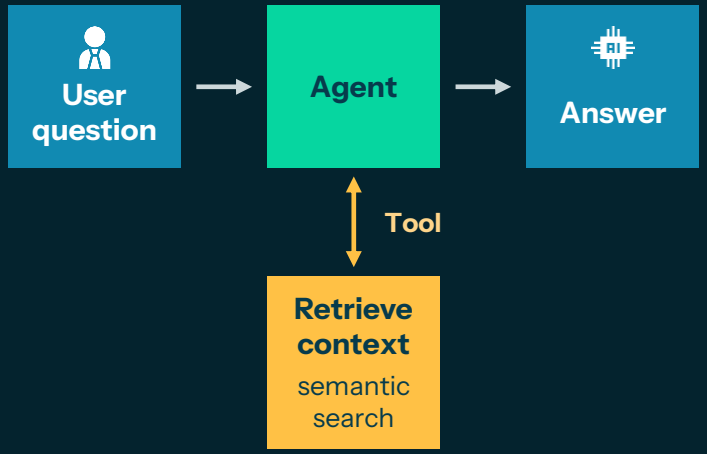
How semantic search works

1. **Chunk** — Split the documents into passages a few hundred words long
2. **Embed** — An embedding model turns each chunk into a vector of numbers
3. **Search** — Embed the question and find the nearest chunks in a vector database

Standard RAG searches once, with the user's question, before the model is called

Agentic RAG

The agent decides when to search and what to search for



Writes its own queries
Rephrases the question or splits it into several searches

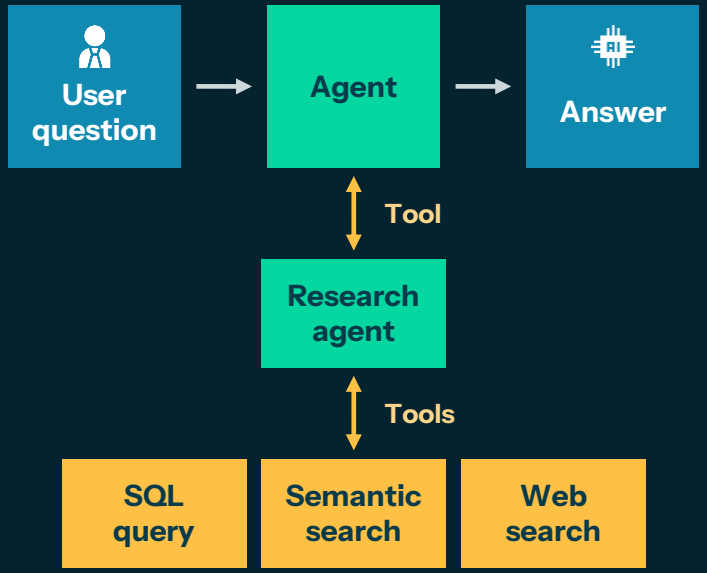
Searches as needed
Follows up on what it finds, or skips the search

Checks the results
Searches again when the passages miss the point

In today's lab, the UnderCover agent searches its memory with a tool

Agentic RAG with a sub-agent

A research agent searches several sources and reports back



Many sources
A database, a vector store and the web

Context isolation
Only the findings come back to the main agent

Deep research
Several research agents can work in parallel

Add the sub-agent when your evals show that it helps

Retrieval evals

Score the search as well as the final answer



Traditional RAG

Test set
Questions paired with the right chunks

Precision
How many retrieved chunks are relevant

Recall
How many relevant chunks were found

Mean Reciprocal Rank (MRR)
How high the first relevant chunk ranks



Agentic RAG

Relevant memories retrieved
Whether the relevant facts came back

The agent's searches
What it searched for and how often

Cost and latency
Each extra search adds tokens and time

In today's lab
`found_fact` on claims that need a fact

RAG, Agentic RAG or fine-tuning?

Three ways to give a model expertise about your business



Traditional RAG

One search with
the user's question

- Simple to build
- Fast and cheap
- Easy to evaluate

- Searches only once
- Can miss the right context



Agentic RAG

The agent searches
as often as it needs

- More powerful retrieval
- More accurate results
- Searches several sources

- Higher cost
- Higher latency



Fine-tuning

Train the model on
your own examples

- Learns a general skill
- Cheaper at inference
- Faster at inference

- Costly and slow to train
- Needs continual retraining

Fine-tuning used to be common but is now rare; but when an agentic harness learns from prior runs by adapting its memory that can loosely be referred to as 'training'

Reprise: Model Context Protocol

An open standard for connecting AI applications to tools and data



A standard way to use someone else's tool
The USB-C port for AI applications



A standard way to share your tool with someone else
Like making an API, but without needing an API



Tools, resources and prompts
What a server can offer, and tools are by far the most used

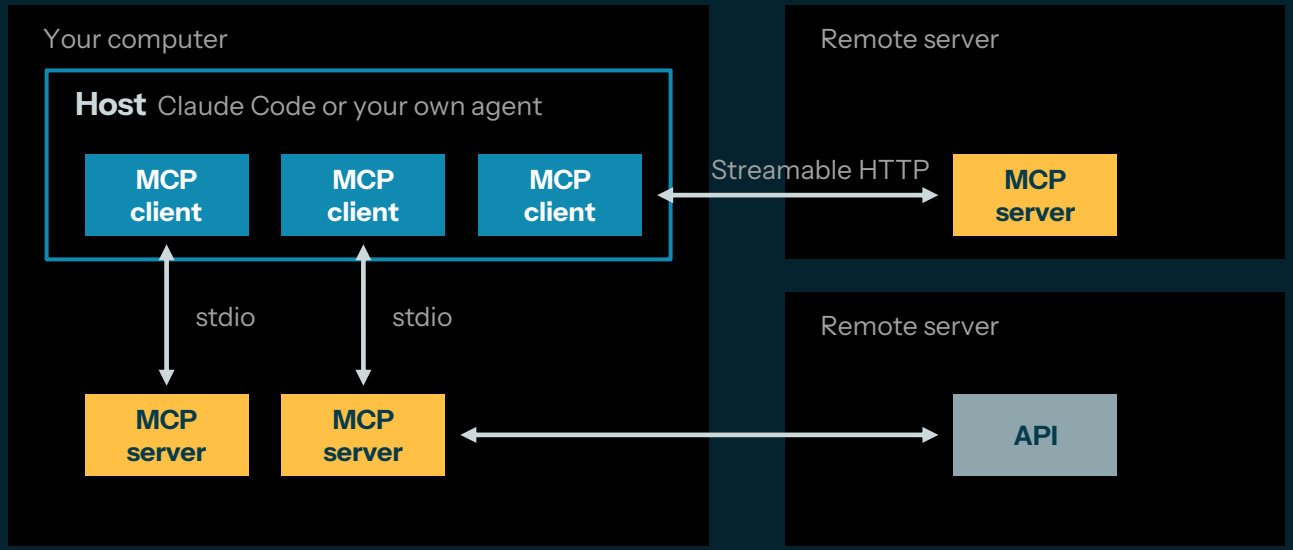


An open standard
Launched by Anthropic in 2024 and given to the Linux Foundation in 2025

The power of MCP is in its adoption

Host, client and server

The host runs one MCP client for each server it uses



Two operations: (1) list your tools and natural language descriptions (2) call a tool

An MCP server is described by parameters

Use them in Claude Code, in ChatGPT or in LangChain



stdio for a local process

The client starts the server locally and talks over standard input and output

memory `{"command": "uvx", "args": ["mcp-server-qdrant"]}`

browser `{"command": "npx", "args": ["@playwright/mcp@latest"]}`



Streamable HTTP for a server at a URL

Replaced the older SSE transport and supports OAuth sign-in

docs `{"url": "https://mcp.context7.com/mcp"}`

Today's lab runs the Qdrant server on your machine over stdio

Three common uses of MCP

Each one connects AI to someone else's tools



Connecting AI apps to integrations

ChatGPT and Claude on web and desktop, with Canva, Figma or Notion



Connecting your own agents to someone else's tools

We do this today with the Qdrant MCP server



Connecting coding agents to new capabilities

As we did with Claude Code and the Playwright browser server

Less common



Building your own MCP server

To share your tools with other people's agents and apps

Benefits and downsides of MCP

What to weigh up before adding an MCP server



Benefits

A huge ecosystem
Most products now publish a server

Works in any host
One server for every MCP application

Supports authentication
Remote servers sign users in with OAuth

Any language
A Python agent can use a Node server



Downsides

Context cost
Every tool definition takes up context

Security risks
Results can carry injected instructions

Uneven quality
Servers vary in quality and upkeep

Sometimes more complex than needed
“Skills” can be simpler for coding agents

If a tool is only for your own agent, there’s usually no need for MCP

Agent memory

An agent needs two kinds of memory:



Short-term memory

The conversation history, sent to the model on every turn



Long-term memory

Saved across sessions in a store the agent searches with a tool

Techniques RAG for semantic search, Markdown files, SQL or custom tools

Four kinds of long-term memory in AWS AgentCore (Week 3)

Semantic
Facts drawn from conversations

Preference
What each user prefers

Summary
A running summary of each session

Episodic
Past tasks and their outcomes

Today's lab: long-term memory with Agentic RAG over MCP

All three of today's ideas in one

Today's lab

In `labs/week2day2.ipynb` and its free twin

1. **The even harder claims**
   25 claims that need the payout tool and additional facts

2. **A memory over MCP**
   The Qdrant MCP server and an assistant that stores and searches facts

3. **Teaching the agent**
   Tell it 33 facts, then ask again in a new conversation

4. **Experiments in Langfuse**
   The payout tool, then the payout tool with memory, on GPT-6 Luna

3 weeks to Forward Deployed Engineer

Theory Practical Projects

Week 3: Become an AI FDE

AI Solutions Architect	Security, Guardrails, API Gateways	Enterprise Context Layer & Integrations	Production Deployment	Capstone

Week 2: Become an AI Architect

Architecture, Evals, Observability	Agentic RAG, MCP, Memory	Durable State & Human in the Loop	Agent Teams, Swarms, Fleet	Self-Improvement, RSI

Week 1: Become an AI Engineer — AI ENGINEER

Path to FDE	Frontier LLMs & Agents	Coding Agents & ADLC	Open-Source, Local, Token Cost	Harness Engineering

AI Forward Deployed Engineer

The AI Architect & FDE Course



47% Complete